GeoLog — Relatório Técnico

UNIPÊ · Tópicos Avançados em Banco de Dados / Arquitetura de Software · Prof. Me. Ricardo Roberto de Lima
Integrantes: _____ . _____ . _____

1. Arquitetura Poliglota

A plataforma aplica **persistência poliglota**: cada tipo de dado é guardado no banco mais adequado. Dados cadastrais e transacionais (motoristas e veículos) exigem integridade referencial e vão para o **SQLite**. A telemetria IoT (GPS, temperatura, velocidade), gerada em alta frequência e com estrutura flexível, vai para o **MongoDB**, que oferece índices geoespaciais nativos. O Streamlit orquestra os dois bancos e os une em memória por meio do `veiculo_id`.

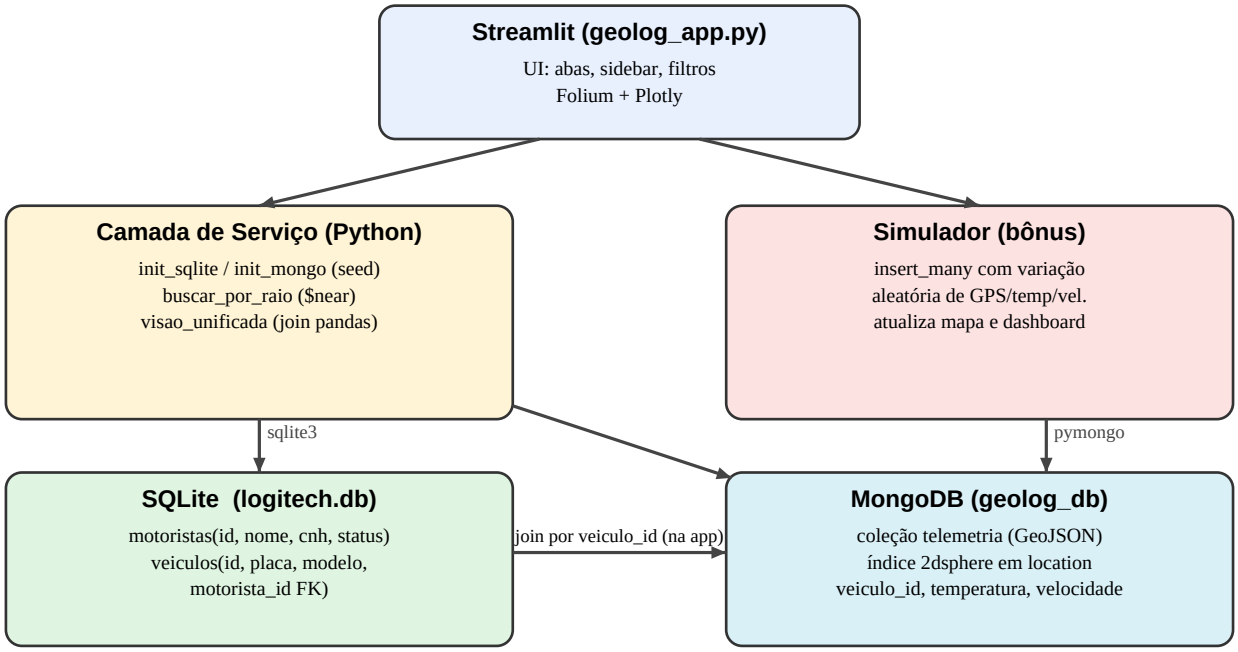

Figura 1 — Diagrama de arquitetura poliglota do GeoLog.

2. Modelagem dos Dados

Banco	Estrutura	Justificativa
SQLite	motoristas(id PK, nome, cnh UNIQUE, status) veiculos(id PK, placa UNIQUE, modelo, motorista_id FK)	Integridade referencial (PRAGMA foreign_keys=ON) e JOINs relacionais.
MongoDB	{ veiculo_id, location:{type:"Point", coordinates:[lon,lat]}, temperatura, velocidade, timestamp }	Documentos flexíveis, escrita rápida e índice 2dsphere para consultas geográficas.

3. Consultas Geoespaciais

Na inicialização, o código cria o índice com `create_index([("location", GEOSPHERE)])`. Para a busca por raio, primeiro um pipeline de agregação (`$sort` + `$group/$first`) obtém o último registro de cada veículo. Em seguida, a consulta `$near` com `$maxDistance` (em metros) filtra esses registros pelo raio escolhido na barra lateral e devolve o resultado já ordenado por proximidade. Vale lembrar que o GeoJSON usa a ordem **[longitude, latitude]**, ao contrário do Folium, que usa [lat, lon].

4. Join Poliglota em Memória

A função `visao_unificada` executa `JOIN veiculos x motoristas` no SQLite, carrega a última telemetria do MongoDB em um DataFrame e cruza os dois com `pandas.merge(on="veiculo_id", how="left")`. O resultado alimenta a tabela (Motorista | Placa | Temperatura | Velocidade | Coordenadas) e os KPIs. O *left join* preserva veículos ainda sem telemetria.

5. Interface e Dashboard

Módulo	Implementação
Mapa (Folium)	Círculo do raio, marcador do ponto de apoio, caminhões dentro do raio (verde; vermelho se > 80 km/h), fora do raio em cinza e trilha (PolyLine) do histórico.
KPIs	Frotas ativas (motorista com status "Ativo"), temperatura média da carga e alertas de velocidade (> 80 km/h).
Gráficos (Plotly)	Linha do histórico de temperatura por veículo e pizza do status dos motoristas.
Bônus	Botão "Simular Movimentação": insere novos pontos com variação aleatória no MongoDB; mapa e dashboard são recalculados a cada interação, sem reiniciar o app.

6. Como Executar

```
pip install streamlit pymongo folium streamlit-folium plotly pandas
docker run -d -p 27017:27017 mongo:7      # ou MongoDB local
streamlit run geolog_app.py               # MONGO_URI opcional
```

7. Conclusão

A separação de responsabilidades ficou clara: o SQLite garante consistência dos cadastros e o MongoDB entrega desempenho e recursos geoespaciais para a telemetria. O custo da abordagem é que o join acontece na aplicação, pois não há chaves estrangeiras entre os bancos. Isso é aceitável aqui, já que o volume de veículos é pequeno. Em produção, seria preciso manter a consistência entre os bancos (por exemplo, validando `veiculo_id` na ingestão).